Contents

Docker & Containers — Deep Dive Notes **1**
1. Docker Architecture . . . 1
2. Image Layers & Union FS . . . 1
3. Multi-Stage Builds (Best Practice) . . . 1
   Python Multi-stage . . . 2
4. BuildKit Features . . . 2
5. Networking . . . 3
   Network Types . . . 3
   Publish Ports . . . 3
6. Volumes & Storage . . . 3
7. Docker Compose (Production-Grade) . . . 3
8. Container Security . . . 5
   Image Security . . . 5
   Runtime Security . . . 5
   Dockerfile Security Checklist . . . 6
   .dockerignore . . . 6
9. Container Runtime Alternatives . . . 6
10. Docker Best Practices Summary . . . 7
 Resources . . . 7

Docker & Containers — Deep Dive Notes

1. Docker Architecture

```
Docker CLI → Docker Daemon (dockerd) → containerd → runc → container
                                     → image management
                                     → network management
                                     → volume management
```

- **dockerd:** API server + orchestration
- **containerd:** Container lifecycle management (OCI-compliant)
- **runc:** OCI runtime — actually creates containers (calls kernel namespaces/cgroups)
- **BuildKit:** Modern build engine (parallel, caching, multi-platform)

2. Image Layers & Union FS

```
Layer 5: COPY app.py /app/          ← writable layer (container)
Layer 4: RUN pip install -r req.txt ← read-only
Layer 3: COPY requirements.txt /    ← read-only
Layer 2: RUN apt-get install -y ... ← read-only
Layer 1: FROM python:3.11-slim      ← base image layers
```

Union FS (OverlayFS): Merges layers into unified view. Copy-on-write: modifications create new layer on top.

```
# See layer history
docker history nginx --no-trunc
docker inspect nginx | jq '.[].RootFS.Layers'
```

3. Multi-Stage Builds (Best Practice)

```
# syntax=docker/dockerfile:1.7

# Stage 1: Builder
FROM golang:1.22-alpine AS builder
WORKDIR /build
COPY go.mod go.sum ./
```

```
RUN --mount=type=cache,target=/go/pkg/mod \
    go mod download
COPY . .
RUN --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 GOOS=linux go build -ldflags="-w -s" -o app ./cmd/main.go

# Stage 2: Final minimal image
FROM gcr.io/distroless/static-debian12
COPY --from=builder /build/app /app
COPY --chown=nonroot:nonroot --from=builder /build/config /config
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/app"]
```

Python Multi-stage

```
FROM python:3.11-slim AS deps
WORKDIR /app
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install --user -r requirements.txt

FROM python:3.11-slim AS runtime
WORKDIR /app
COPY --from=deps /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
RUN useradd -r -u 1001 appuser && chown -R appuser:appuser /app
USER appuser
EXPOSE 8080
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:8080", "app:application"]
```

4. BuildKit Features

```
# Enable BuildKit
export DOCKER_BUILDKIT=1

# Build with caching
docker buildx build \
  --cache-from type=registry,ref=registry.io/myapp:cache \
  --cache-to type=registry,ref=registry.io/myapp:cache,mode=max \
  --platform linux/amd64,linux/arm64 \
  --push \
  -t registry.io/myapp:latest .

# Build secrets (don't bake into image)
docker buildx build \
  --secret id=npmrc,src=$HOME/.npmrc \
  --secret id=pip_index,env=PIP_INDEX_URL \
  .

# In Dockerfile:
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc \
    npm install
```

5. Networking

Network Types

```
# Bridge (default) — isolated network, NAT to host
docker run --network bridge nginx

# Host — container uses host network directly, fastest
docker run --network host nginx

# None — completely isolated
docker run --network none myapp

# Custom bridge — DNS by container name
docker network create --driver bridge mynet
docker run --network mynet --name db postgres
docker run --network mynet app  # can reach "db" by name

# Overlay — multi-host (Swarm/K8s)
docker network create --driver overlay --attachable myoverlay
```

Publish Ports

```
docker run -p 8080:80           # host:container (all interfaces)
docker run -p 127.0.0.1:8080:80 # bind to localhost only
docker run -P                    # publish all EXPOSE'd ports to random
```

6. Volumes & Storage

```
# Named volume (managed by Docker, persistent)
docker run -v mydata:/var/lib/postgresql/data postgres
docker volume ls; docker volume inspect mydata

# Bind mount (host path, development)
docker run -v $(pwd)/src:/app/src:ro myapp

# tmpfs (in-memory, not persisted)
docker run --tmpfs /tmp:size=100m,mode=1777 myapp

# Volume drivers (for NFS, cloud storage)
docker volume create --driver local \
  --opt type=nfs \
  --opt o=addr=nfs-server.example.com,rw \
  --opt device=:/exports/data \
  nfs-data
```

7. Docker Compose (Production-Grade)

```
# docker-compose.yml
version: "3.9"

x-common-env: &common-env
  TZ: UTC
  LOG_LEVEL: info

x-logging: &default-logging
  driver: json-file
```

```yaml
  options:
    max-size: "10m"
    max-file: "5"

services:
  api:
    image: myapp:${IMAGE_TAG:-latest}
    build:
      context: .
      dockerfile: Dockerfile
      target: runtime
      cache_from: [myapp:latest]
    restart: unless-stopped
    ports:
      - "127.0.0.1:8080:8080"    # localhost only
    environment:
      <<: *common-env
      DATABASE_URL: postgresql://user:${DB_PASS}@db:5432/app
    env_file: [.env]
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks: [backend, frontend]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 512M
    logging: *default-logging
    security_opt:
      - no-new-privileges:true
    read_only: true
    tmpfs:
      - /tmp
      - /var/run

  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: app
      POSTGRES_USER: user
      POSTGRES_PASSWORD: ${DB_PASS}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d app"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks: [backend]
```

```yaml
    logging: *default-logging

  redis:
    image: redis:7-alpine
    restart: unless-stopped
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
    networks: [backend]

  nginx:
    image: nginx:alpine
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./certs:/etc/nginx/certs:ro
    networks: [frontend]
    depends_on: [api]

networks:
  backend:
    driver: bridge
    internal: true    # no internet access
  frontend:
    driver: bridge

volumes:
  pgdata:
    driver: local
```

8. Container Security

Image Security

```bash
# Scan for vulnerabilities
trivy image myapp:latest
trivy image --severity HIGH,CRITICAL myapp:latest
grype myapp:latest

# Use minimal base images
# Distroless (Google): no shell, no package manager, minimal attack surface
FROM gcr.io/distroless/java17-debian12
# Alpine: musl libc, small (~5MB)
FROM alpine:3.19
# Scratch: empty (for Go static binaries)
FROM scratch
```

Runtime Security

```bash
# Never run as root
docker run --user 1001:1001 myapp

# Read-only root filesystem
docker run --read-only --tmpfs /tmp myapp
```

```
# Drop capabilities
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE nginx

# No privilege escalation
docker run --security-opt=no-new-privileges myapp

# Seccomp profile (restrict syscalls)
docker run --security-opt seccomp=profile.json myapp

# AppArmor profile
docker run --security-opt apparmor=docker-default myapp
```

Dockerfile Security Checklist

```
# 1. Use specific version tags (not :latest)
FROM python:3.11.8-slim

# 2. Run as non-root
RUN useradd -r -u 1001 -g 0 appuser
USER 1001

# 3. Don't store secrets
# BAD:  ENV AWS_SECRET_KEY=abc123
# GOOD: Read at runtime from env or secrets manager

# 4. Multi-stage to exclude build tools
# 5. Minimize layers
# 6. Use .dockerignore
# 7. HEALTHCHECK instruction
# 8. Verify downloads with checksums
RUN wget -q https://example.com/binary.tar.gz && \
    echo "sha256sum:expected  binary.tar.gz" | sha256sum -c
```

.dockerignore

```
.git
.gitignore
**/*.test
**/*.spec
docs/
node_modules/
*.md
.env
*.log
__pycache__
*.pyc
.pytest_cache
.coverage
```

9. Container Runtime Alternatives

Runtime	Description
containerd	Industry standard (used by K8s)
CRI-O	Lightweight K8s-specific runtime
gVisor	Sandboxed runtime (adds kernel layer)
Kata Containers	VM-level isolation with container UX

Runtime	Description
Podman	Rootless, daemonless Docker alternative

10. Docker Best Practices Summary

1. Use multi-stage builds — keep production images small
2. Pin image versions — never use `latest` in production
3. Run as non-root user
4. One process per container (single responsibility)
5. Use `COPY` not `ADD` unless you need URL/tar features
6. Use `CMD` for default args, `ENTRYPOINT` for executable
7. Set `HEALTHCHECK` in Dockerfile
8. Always use `.dockerignore`
9. Sort multi-line RUN commands alphabetically (readability + layer caching)
10. Use BuildKit with `--mount=type=cache` for faster builds

Resources

- https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
- https://docs.docker.com/build/buildkit/
- https://github.com/GoogleContainerTools/distroless
- https://aquasecurity.github.io/trivy/
- https://www.youtube.com/@TechWorldwithNana — Docker tutorials